

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 5

Searching



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

Genap 2024/2025

A. Tujuan Praktikum

Agar setiap mahasiswa tahu cara mengaplikasikan struktur data Searching dalam python. Dan mahasiswa mampu memahami algoritma struktur data searching, macam-macam penerapan, dan fungsi pada setiap code

B. Dasar Teori

Algoritma pencarian adalah alat penting dalam ilmu komputer yang digunakan untuk menemukan item tertentu dalam kumpulan data. Algoritma ini dirancang untuk menavigasi struktur data secara efisien untuk menemukan informasi yang diinginkan, mejadikan dasarnya dalam berbagai aplikasi seperti data bases, mesi pencarian web

Linier Search

Linier search didefinisikan sebagai algoritma pencarian berurutan yang dimulai dari satu ujung dan menelusuri setiap elemen daftar hingga elemen yang diinginkan ditemukan, jika tidak, pencarian akan berlanjut hingga akhir kumpulan data.

Cara kerja Linier search

- Setiap elemen dianggap sebagai potensi kecocokan untuk kunci dan diperiksa kesamaannya
- Jika ada elemen yang ditemukan sama dengan kuncinya, pencarian berhasil dan ideks elemen tersebut dikembalikan
- Jika tidak ada elemen yang ditemukan sama dengan kuncinya pencarian akan menghasilakn” Tidak ditemukan kecocokan”

Komplesitas Waktu

- Kasus terbaik : dalam kasus terbaik, kuncinya mungkin ada di indeks pertama. Jadi kompleksitas kasus terbainya adalah $O(1)$
- Kasus Terburuk : Dalam kasus terburuk, kuncinya mungkin ada pada indeks terakhir yakni berlawanan dengan akhir pencarian dalam dafta. Jadi kompleksitas kasus terburuknya adalah $O(N)$ dangan N adalah ukuran daftar
- Kasus Rata-rata : $O(N)$
- Ruang tambahan : $O(1)$ karena kecuali variable yang akan diinterasi melalui daftar, tidak ada variable lain yang digunakan

Kekurangan Pencarian Linier

- Pencarian linier memilik kompleksitas waktu $O(N)$, yang pada giliranya membantunya lambat untuk kumpulan data besar

- Tidak cocok untuk array besar

Kelebihan Linier Search

- Pencarian linier dapat digunakan terlepas dari apakah array dirutkan atau tidak. Ini dapat digunakan pada array dengan tipe data apapun.
- Tidak memerlukan memori tambahan
- Ini adalah algoritma yang cocok untuk kumpulan data kecil

Binary Search

Binary search didefinisikan sebagai algoritma pencarian yang digunakan dalam array yang dirutkan dengan membagi interval pencarian menjadi dua berulang. Ide pencarian biner adalah menggunakan informasi bahwa array telah dirutkan dan mengurangi kompleksitas waktu menjadi $O(\log N)$.

Algoritma binary search dapat diimplementasikan dengan dua cara

1. Algoritma Pencarian Biner Iteratif

Disini kita menggunakan perulangan while untuk melanjutkan proses membandingkan kunci dan membagi ruang pencarian menjadi dua bagian

Kompleksitas waktu : $O(\log N)$

Ruang tambahan : $O(1)$

2. Algoritma Pencarian Biner Rekursif

Buat fungsi rekursif dan bandingkan bagian tengah ruang pencarian dengan kunci. Dan berdasarkan hasilnya, kembalikan indeks tempat kunci ditemukan atau panggil fungsi rekursif untuk ruang pencarian berikutnya

Kompleksitas waktu :

- Kasus terbaik : $O(1)$
- Kasus rata-rata : $O(\log N)$
- Kasus Terburuk $O(\log N)$

Ruang tambahan : $O(1)$, jika tumpukan panggilan rekursif dipertimbangkan maka ruang tambahannya adalah $O(\log N)$

Keuntungan Pencarian Biner

- Pencarian biner lebih cepat daripada pencarian linier, terutama untuk array besar
- Lebih efisien dibandingkan algoritma pencarian lain dengan kompleksitas waktu serupa, seperti pencarian interpolasi atau pencarian eksponensial
- Pencarian biner sangat cocok untuk mencari kumpulan data besar yang disimpan di emmemo eksternal, seperti di hard drive atau cloud

Kekurangan Pencarian Biner

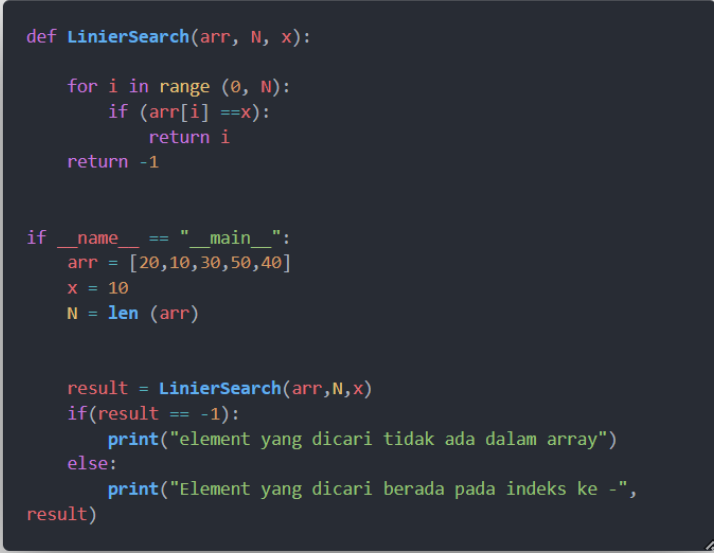
- Array harus diurutkan
- Pencarian biner mengharuskan struktur data yang dicari disimpan di lokasi memori yang berdekatan
- Pencarian biner mengharuskan elem-elemen array dapat dibandingkan, artinya elemn-elemen tersebut harus diurutkan

Implementasi Pencarian Biner

- Pencarian biner dapat digunakan sebagai bahan penyusun untuk algoritma yang lebih kompleks yang digunakan dalam pembelajaran mesin, seperti algoritma untuk melatih jaringan saraf atau menemukan hyperparameter optimal untuk suatu model
- Ini dapat digunakan untuk pencarian dalam grafik komputer seperti algoritma untuk ray tracing atau pemetaan tekstur
- Dapat digunakan untuk mencari database

C. Hasil dan Pembahasan

1. Linier Search



```
def LinierSearch(arr, N, x):  
    for i in range (0, N):  
        if (arr[i] ==x):  
            return i  
    return -1  
  
if __name__ == "__main__":  
    arr = [20,10,30,50,40]  
    x = 10  
    N = len (arr)  
  
    result = LinierSearch(arr,N,x)  
    if(result == -1):  
        print("element yang dicari tidak ada dalam array")  
    else:  
        print("Element yang dicari berada pada indeks ke -",  
result)
```

Gambar 1.1

- Def LinierSearching ini mengimplementasikan algoritma pencarian linier. Ini menerima 3 parameter arr atau daftar elemen yang akan dicari, N Panjang array, dan x elemen yang akan dicari dalam array
- Melalui array menggunakan loop for dan jika elemen pada indek ke I dari arry sama dengan x maka fungsi ini mengembalikan indek ke i. Apabila tidak ditemukan elemen yang sama denganx fungsi mengembalikan ke-1
- If name ini akan dijalankan hanya jika skrip ini dieksekusi langsung
- Arr di atas adalah value yang ada didalam array tersebut
- X = 10 adalah menentukan elemen yang akan dicari yaitu 10
- Result untuk memanggil fungsi liniarserching dengan parameter yang sesuia dan menyimpan hasilnya dalam variable result
- If(result == -1) untuk menegecek apakah hasil pencarian sama dengan -1, yang menandakan bahwa elemen tidak ditemukan
- Else untuk akan mencetak pesan dengan indeks tempat elemen tersebut ditemukan

2. Binary Search Iteratif

```
def binarysearch (arr,l,r,x):  
    while l <=r:  
        mid = l +(r-l)//2  
        if arr[mid] == x:  
            return mid  
        elif arr[mid] < x :  
            l = mid +1  
        else :  
            r = mid -1  
    return -1  
  
if __name__=="__main__":  
    arr = [2,5,8,12,16,23,38,56,72,91]  
    x = 72  
  
    result = binarysearch(arr,0,len(arr)-1,x)  
  
    if result !=-1:  
        print("element is present at index",result)  
    else:  
        print("element is not present in array")
```

Gambar 1.2

- Fungsi binary diatas adalah untuk mencari elemen x dalam array yang sudah diurutkan
- Pencarian dilakukan dengan membagi array menjadi 2 bagian dan membandingkan elemen tengah dengan x
- Jika elemen tengah sama dengan x maka fungsi mengembalikan indeksinya
- Jika elemen tengah kurang dari x, maka pencarian dilakukan pada setengah kanan array
- Jika elemen tengah lebih dari x, maka pencarian dilakukan pada setengah kiri array
- Untuk memanggil fungsi array harus diurutkan terlebih dahulu
- Fungsi binary dipanggil dengan parameter sesuai, yaitu array, indeks awal, indeks akhir dan elemen yang dicari
- Hasil pencarian disimpan dalam variable result
- Jika hasil pencarian result tidak sama dengan -1, maka mencetak pesan bahwa elemen ditemukan berserta indeksinya
- Jika hasil pencariannya sama dengan -1, maka mencetak pesan bahwa elemen tidak ditemukan dalam array

3. Binary Search Rekursif

```
def binarysearch (arr,l,r,x):  
    if r >= 1:  
        mid = 1 + (r-1)//2  
        if arr[mid]== x:  
            return mid  
        elif arr[mid]> x:  
            return binarysearch(arr,l,mid-1,x)  
        else:  
            return binarysearch(arr,mid+1,r,x)  
    else :  
        return -1  
  
if __name__ == "__main__":  
    arr = [2,5,8,12,16,23,38,56,72,91]  
    x = 72  
  
    result = binarysearch(arr,0,len(arr)-1,x)  
  
    if result != -1:  
        print("elemenet terdapat pada indeks ke -",result)  
    else:  
        print("elemenet tidak ada dalam array")
```

Gambar 1.3

- Fungsi binary searching rekursif ini mencari elemen x dalam array sudah diurutkan
- Ini menggunakan pendekatan rekursif
- Jika indeks r msaih lebih besar atau sama dengan indeks -1 maka menghitung indek tengah dari rentang pencarian
- Jika elemen tengah sama dengan x maka mengembalikan indeks tersebut
- Jika elemen tengah lebih besar dari x,melakukan pencarian pada setengah kiri array
- Jika elem tengah lebih kecil dari x, melakukan pencarian pada setengah kanan array
- Proses ini dulangi secara rekursif sampai elemen sitemukan atau rentang pencarian tidak dapat dibagi bagi lagi
- Jika r kurang dari 1 mengembalikan -1
- Fungsi binary searching ini untuk memanggil dengan parameter yang sesuia , yaitu array, indeks awal, indeks akhir, dan elemene yang dicari x
- Dan hasil pencarian akan disimpan dalam variable result
- Jika hasil pencarian tidak sama dengan – 1, maka mencetak pesan bahwa elemen ditemukan berserta indeksnya
- Jika hsilnya pencarian sama dengan -1, maka mencetak pesan bahwa elemen tidak ditemukan dalam array

4. Jump Search

```
import math
def jumpSearch(arr,x,n):

    step = math.sqrt(n)

    prev = 0
    while arr[int(min(step,n)-1)]< x:
        prev = step
        step += math.sqrt(n)
        if prev >= n:
            return -1

    while arr[int(prev)]<x:
        prev +=1

    if prev == min(step,n):
        return -1

    if arr[int(prev)]==x:
        return prev

    return -1

arr=[0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610]
x = 55
n = len(arr)

index = jumpSearch(arr,x,n)
print("Number",x,"is at index", "%0f"%index)
```

Gambar 1.4

- pertama import math untuk menggunakan fungsi sqrt atau akar kuadrat
- fungsi jumpsearching menerima 3 parameter array,x, dan n
- `step = math.sqrt(n)` untuk menghitung Panjang langkah menggunakan akar kuadrat dari panjang array n
- `prev =0` untuk inisialilasi variable prev sebagai indeks awal
- melakukan loop while selama ini elemen di indeks step kurang dari x
- memperbarui nilai prev dengan nilai step
- menambahkan nilai step dengan akar kuadrat dari Panjang array n
- jika nilai prev sudah melebihi atau sama dengan Panjang array n, maka mengembalikan -1
- Setelah menemukan rentang yang mungkin mengandung elemen x, dilakukan pencarian linier menggunakan loop while selama nilai elemen di indeks prev kurang dari x maka menambahkan nilai prev sejajar dengan penacrian linier
- Jika prev sudah mencapai batas `min(step,n)` mengebalikna-1
- Array di atas berisi deret Fibonacci yang dirutukan
- Elem yang dicari adalah 55
- Fungsi jumpsearch dipanggil dengan parameter yang sesuai
- Hasil pencarian disimpan dalam variable indeks
- Mencetak pesan hasil pencarian berserta indeksnya

D. Studi Kasus

1. Studi Kasus 1

```
# Python program of above implementation
# Structure is used to return two values from minMax()
class pair:
    def __init__(self):
        self.min = 0
        self.max = 0
def getMinMax(arr: list, n: int) -> pair:
    minmax = pair()

    if n == 1:
        minmax.max = arr[0]
        minmax.min = arr[0]
        return minmax
# If there are more than one elements, then initialize min
# and max
    if arr[0] > arr[1]:
        minmax.max = arr[0]
        minmax.min = arr[1]
    else:
        minmax.max = arr[1]
        minmax.min = arr[0]
    for i in range(2, n):
        if arr[i] > minmax.max:
            minmax.max = arr[i]
        elif arr[i] < minmax.min:
            minmax.min = arr[i]
    return minmax

if __name__ == "__main__":
    arr = [1000, 11, 445, 1, 330, 3000]
    arr_size = 6
    minmax = getMinMax(arr, arr_size)
    print("Minimum element is", minmax.min)
    print("Maximum element is", minmax.max)

# This code is contributed by
# sanjeev2512
```

- Mendefinisikan kelas pair untuk menyimpan dua nilai, yaitu minimum dan maksimum
- Fungsi get mind untuk menerima 2 parameter arr dan n
- Jika Panjang array n sama dengan 1 maka nilai minimum dan maksimum dengan eleme tunggal tersebut
- Jika Panjang array n lebih dari 1 maka nilai minimum dan maksimum dengan 2 elemen pertama dalam array
- Melalui array untuk memperbarui nilai minimum dan maksimum sesuai dengan elemen elemen selanjutnya
- Pemanggilan array berisi elemen-elemen yang akan dicari nilai minimum dan maksimumnya
- Pnajang array diatas adalah 6
- Fungsi get getminmax dipanggil dengan parameter yang sesuai
- Nilai minimum dan maksimum disimpan dalam variable minmax
- Mencetak nilai minimum dan maksimum

2. Studi Kasus 2

```
def printTwoElements(arr):  
    n = len(arr)  
    temp = [0] * n # Creating temp array of size n with initial values as e.  
    repeatingNumber = -1  
    missingNumber = -1  
  
    for i in range(n):  
        temp[arr[i] - 1] += 1  
        if temp[arr[i] - 1] > 1:  
            repeatingNumber = arr[i]  
    for i in range(n):  
        if temp[i] == 0:  
            missingNumber = i + 1  
            break  
    print("The repeating number is", repeatingNumber, ".")  
    print("The missing number is", missingNumber, ".")  
  
arr = [7, 3, 4, 5, 5, 6, 2]  
printTwoElements(arr)
```

- Fungsi diatas untuk mencari 2 elemen dalam array yang memiliki dua kondisi
- Yang satu muncul lebih dari sekali dan satu elemen tidak ada dalam array
- Pertama inisialisasi variable $n = \text{len}(\text{arr})$ untuk mendapatkan Panjang array
- Temp untuk membuat array sementara dengan panjang n dan meninisialisasi semua elemen dengan nilai 0
- Repeating number dan missing number untuk menyimpan nilai elemen yang muncul lebih dari sekali dan nilai elemen yang tidak ada dalam array
- Pada repeating number melalui array dengan variable i dan pada setiap nilai indeks $\text{arr}[i] - 1$ dalam array sementara temp
- Jika nilai di indeks tersebut lebih dari 1, maka elemen $\text{arr}[i]$ dianggap sebagai elemen yang muncul lebih dari sekali, dan nilai ini disimpan dalam repeatingNumber
- Untuk pencarian missing number atau yang tidak ada didalam array
- Jika nilai indeks i adalah 0, maka elemen $(i+1)$ dianggap sebagai elemen yang tidak ada dalam array, dan nilai ini disimpan dalam missingNumber
- Jika sudah menemukan nilai missingNumber maka akan keluar dari looping

3. Studi Kasus 3

```
def pasangan (arr, numb):
    status = False
    for x in range (len(arr)):
        for y in range (len(arr)):
            if arr[x] != arr[y]:
                if (arr[x] + arr[y]) == numb:
                    status = True
    print (status)

arr = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]
nol = 0
print('Cek pasangan dengan jumlah ')
pasangan (arr, nol)
tigasatu= 31
print('Cek pasangan dengan jumlah 31')
pasangan (arr, tigasatu)
duaempat=24
print('Cek pasangan dengan jumlah 24')
pasangan (arr, duaempat)
```

- Fungsi pasangan diatas digunakan untuk mencari pasangan elemen dalam array yang memiliki jumlah tertentu.Selanjutnya akan dilakukan beberapa pengujian dengan menggunakan array yang telah didefinisikan sebelumnya
- Fungsi pasangan menerima 2 parameter array dan numb
- Menggunakan dua loop for untuk melalui semua pasangan elemen dalam array
- Jika terdapat pasangan elemen yang jumlahnya sama dengan numb, mengubah variable status menjadi true
- Pada gambar diatas bahwa array berisikan nilai dari 1 sampai 20
- Pemanggilan fungsi pasangan dengan beberapa nilai yang berbeda nol,tigasatu,duaempat
- Output mencetak true jika terdapat pasangan elemen dengan jumlah sesuai dengan jumlah nilai yang dicari dan false jika tidak ditemukan

E. Kesimpulan

Kesimpulan dari praktikum di atas adalah mengetahui konsep konsep dasar terkait struktur data pencarian dan bagaimana cara menggunakan nya juga pada linier search dan binary search yang kita pelajari sebelumnya.

F. Referensi

- <https://www.geeksforgeeks.org/searching-algorithms/>